Tabla de contenidos

1. *Lock* como atributo de una subclase de `Thread`
2. Patrón productor-consumidor
    a. `Queue`
3. Simulaciones

Lock como atributo de una subclase de `Thread`

Como recordarás, una manera de crear nuestros *threads* consiste en crear una clase que herede de `Thread` y sobreescribir el método `run` e `__init__`. Podemos aprovechar de colocar los *locks* que necesitemos **como atributo de clase**. De esta manera, tendremos acceso a un mismo *lock* para todos los *threads* de nuestra clase y organizaremos mejor nuestro código.

Paréntesis: atributos de clase y atributos de instancia

Al usar clases, normalmente definimos atributos para las instancias mediante el uso de `self.atributo = valor`. Esto genera la creación de un **atributo de instancia**, cuyo valor es accesible desde cualquier punto de la instancia mediante `self`. Si definimos un atributo al nivel de los métodos, se le conoce como **atributo de clase** y es accesible por todas las instancias mediante el nombre de la clase: `Clase.atributo`. También, es posible acceder mediante `self` en una instancia, siempre y cuando no tenga un atributo de instancia del mismo nombre definido. Cuando se busca un valor de atributo mediante `self.atributo`, el orden de búsqueda es:

1. Buscar en la instancia. (atributo de instancia)
2. Si no existe, buscar en la clase. (atributo de clase)
3. Si no existe, error (`AttributeError`).

Ejemplo

En el siguiente ejemplo, escribiremos en un mismo archivo `txt` desde varios *threads*.

```
import threading
import time
import os
from random import random
import typing


class EscritorArchivo(threading.Thread):
    """
    Esta clase modela un thread. Dentro creamos un objeto para bloqueo
dentro de la clase.
    El Lock es una variable independiente de cada thread y es común
para todas las instancias.
    """
```

```
    # Atributo de clase
    # Accesible mediante Clase.lock o self.lock desde una instancia
    lock = threading.Lock()

    def __init__(self, numero: int, archivo: typing.TextIO) -> None:
        # Mandamos un nombre a la clase padre con el atributo name
        super().__init__(name=f"EscritorArchivo número {numero}")
        self.numero = numero
        self.archivo = archivo

    def run(self) -> None:
        print(f"[{self.name}] ¡Comenzó a trabajar!")
        for _ in range(self.numero):
            with self.lock:  # Acceso al lock
                self.archivo.write(f"Línea escrita por # {self.name}\
n")
                print(f"[{self.name}] ¡escribió una línea!")
            # Hacemos que se demore una cantidad random uniforme [0,
1)
            time.sleep(random())


# Creamos un archivo para escribir una salida
# Luego creamos los threads que escribirán dentro del archivo
with open(os.path.join("files", "salida.txt"), "w", encoding="utf-8")
as archivo:
    # Creamos los threads
    cantidad_threads = 7

    threads = []
    for i in range(1, cantidad_threads + 1):
        threads.append(EscritorArchivo(i, archivo))

    # Hacemos partir los threads
    for thread in threads:
        thread.start()

    # Esperamos a todos los threads antes de cerrar el archivo
    for thread in threads:
        thread.join()

    print("Todos los threads terminaron. Revise el resultado en
files/salida.txt")
```

```
[EscritorArchivo número 1] ¡Comenzó a trabajar![EscritorArchivo número
2] ¡Comenzó a trabajar!
[EscritorArchivo número 2] ¡escribió una línea!

[EscritorArchivo número 1] ¡escribió una línea!
```

```
[EscritorArchivo número 3] ¡Comenzó a trabajar!
[EscritorArchivo número 3] ¡escribió una línea!
[EscritorArchivo número 4] ¡Comenzó a trabajar!
[EscritorArchivo número 4] ¡escribió una línea!
[EscritorArchivo número 5] ¡Comenzó a trabajar!
[EscritorArchivo número 5] ¡escribió una línea!
[EscritorArchivo número 6] ¡Comenzó a trabajar!
[EscritorArchivo número 6] ¡escribió una línea!
[EscritorArchivo número 7] ¡Comenzó a trabajar!
[EscritorArchivo número 7] ¡escribió una línea!
[EscritorArchivo número 6] ¡escribió una línea!
[EscritorArchivo número 7] ¡escribió una línea!
[EscritorArchivo número 3] ¡escribió una línea!
[EscritorArchivo número 2] ¡escribió una línea!
[EscritorArchivo número 7] ¡escribió una línea!
[EscritorArchivo número 4] ¡escribió una línea!
[EscritorArchivo número 5] ¡escribió una línea!
[EscritorArchivo número 6] ¡escribió una línea!
[EscritorArchivo número 3] ¡escribió una línea!
[EscritorArchivo número 4] ¡escribió una línea!
[EscritorArchivo número 4] ¡escribió una línea!
[EscritorArchivo número 7] ¡escribió una línea!
[EscritorArchivo número 5] ¡escribió una línea!
[EscritorArchivo número 7] ¡escribió una línea!
[EscritorArchivo número 5] ¡escribió una línea!
[EscritorArchivo número 6] ¡escribió una línea!
[EscritorArchivo número 7] ¡escribió una línea!
[EscritorArchivo número 6] ¡escribió una línea!
[EscritorArchivo número 5] ¡escribió una línea!
[EscritorArchivo número 6] ¡escribió una línea!
[EscritorArchivo número 7] ¡escribió una línea!
Todos los threads terminaron. Revise el resultado en files/salida.txt
```

Patrón productor-consumidor

Un problema común en programación concurrente es el patrón **productor-consumidor**. Este se origina cuando dos o más *threads*, conocidos como **productores** y **consumidores**, acceden a un mismo espacio de almacenamiento o ***buffer***.

Bajo este esquema, los productores ponen ítems en el *buffer* y los consumidores sacan elementos del *buffer*. Este modelo permite la comunicación entre distintos *threads*. Por lo general el *buffer* compartido en este modelo se implementa mediante una **cola sincronizada** o una **cola *thread-safe***, que funciona correctamente al ser usada por múltiples *threads*.

Si bien los `deque` permiten agregar y sacar elementos desde ambos extremos en forma segura con *threads*, **nada nos asegura** que si vimos que había un objeto para sacar, ese objeto todavía esté cuando queramos sacarlo. Por lo tanto, tenemos que asegurarnos nosotros mismos – vía *locks* – de que revisar si había algo y sacarlo sea una operación atómica.

Por ejemplo, supongamos que el productor es un panadero, y los consumidores son clientes de un supermercado. El panadero colocará piezas de pan cuando las tenga listas, y los consumidores sacarán estas piezas cuando estén disponibles. Implementemos esto:

```
from collections import deque
from random import choices
import threading
import time


piezas_de_pan = deque()


def panadero() -> None:
    # El panadero hará 3 veces pan
    for partida in range(3):
        # En cada vez, producirá 5 piezas de pan
        # Se demorará 5 segundos por vez (que rápido :D)
        time.sleep(5)
        piezas = choices(["Marraqueta", "Baguette", "Hallulla"], k=5)
        print("[Panadero] Produje 5 piezas de pan en la partida",
partida)
        piezas_de_pan.extend(piezas)


lock_sacar_pan = threading.Lock()


def cliente(i: int) -> None:
    print(f"[Cliente {i}] ¡Quiero pan!")
    while True:
        # El cliente verifica si hay pan antes de sacarlo
        # Necesitamos asegurarnos que si vio que había pan, nadie se
lo quite
        # Para eso, ponemos un lock para que la operación
        # de ver si había pan y luego sacarlo sea atómica
        with lock_sacar_pan:
            if piezas_de_pan:
                mi_pan = piezas_de_pan.popleft()
                print(f"[Cliente {i}] ¡Saqué mi {mi_pan}!")
                # Cuando logre sacar su pan, deja de verificar y
termina
                break


thread_panadero = threading.Thread(target=panadero)
threads_clientes = []

for i in range(15):
    threads_clientes.append(threading.Thread(target=cliente,
```

```
args=(i, )))

thread_panadero.start()
for thread_cliente in threads_clientes:
    thread_cliente.start()

# Esperamos a que todos los threads finalicen para que esta celda
termine con su ejecución
for thread_cliente in threads_clientes:
    thread_cliente.join()
```

```
[Cliente 0] ¡Quiero pan!
[Cliente 1] ¡Quiero pan!
[Cliente 2] ¡Quiero pan!
[Cliente 3] ¡Quiero pan!
[Cliente 4] ¡Quiero pan!
[Cliente 5] ¡Quiero pan!
[Cliente 6] ¡Quiero pan!
[Cliente 7] ¡Quiero pan!
[Cliente 8] ¡Quiero pan!
[Cliente 9] ¡Quiero pan!
[Cliente 10] ¡Quiero pan!
[Cliente 11] ¡Quiero pan!
[Cliente 12] ¡Quiero pan!
[Cliente 13] ¡Quiero pan!
[Cliente 14] ¡Quiero pan!
[Panadero] Produje 5 piezas de pan en la partida 0
[Cliente 1] ¡Saqué mi Hallulla!
[Cliente 12] ¡Saqué mi Marraqueta!
[Cliente 14] ¡Saqué mi Hallulla!
[Cliente 3] ¡Saqué mi Marraqueta!
[Cliente 5] ¡Saqué mi Hallulla!
[Panadero] Produje 5 piezas de pan en la partida 1
[Cliente 2] ¡Saqué mi Baguette!
[Cliente 9] ¡Saqué mi Baguette!
[Cliente 7] ¡Saqué mi Marraqueta!
[Cliente 11] ¡Saqué mi Baguette!
[Cliente 13] ¡Saqué mi Hallulla!
[Panadero] Produje 5 piezas de pan en la partida 2
[Cliente 0] ¡Saqué mi Hallulla!
[Cliente 6] ¡Saqué mi Marraqueta!
[Cliente 8] ¡Saqué mi Marraqueta!
[Cliente 10] ¡Saqué mi Hallulla!
[Cliente 4] ¡Saqué mi Marraqueta!
```

La implementación anterior es correcto, pero tiene un problema de eficiencia: mientras los clientes esperan, utilizan tiempo de CPU en forma innecesaria verificando si hay pan o no, ya que si no hay pan vuelven a revisar de inmediato. Podríamos evitar este uso innecesario de tiempo si el panadero enviara una señal a los clientes cuando él tenga pan listo. No obstante, los clientes

tendrán que verificar de igual manera que todavía quede pan, porque podría haber más clientes que piezas de pan disponibles en ese momento.

Afortunadamente, en Python existe una biblioteca optimizada para manejar este tipo de casos.

`Queue`

El módulo `queue` tiene implementada una cola hecha para situaciones donde hay varios *threads*. Tiene métodos que la hacen un poco diferente a la implementada en `collections`:

- `put()`: Agrega un ítem al final de la cola (*push*)
- `get()`: Remueve y retorna un ítem de la cola (*pop*). Lo interesante es que este método **espera** hasta que exista algo para sacar de la cola.
- `task_done()`: Requiere ser llamado cada vez que un ítem extraído de la cola ha sido procesado.
- `join()`: El *thread* que llame a este método queda en pausa hasta que todos los ítems de la cola hayan sido procesados.

Volvamos al ejemplo anterior, ahora usando `Queue`.

```
from queue import Queue
from random import choices
import threading
import time


piezas_de_pan = Queue()


def panadero() -> None:
    # El panadero hará 3 veces pan
    for partida in range(3):
        # En cada vez, producirá 5 piezas de pan.
        # Se demorará 5 segundos por vez (que rápido :D)
        time.sleep(5)
        print("[Panadero] Produje 5 piezas de pan en la partida",
partida)
        piezas = choices(["Marraqueta", "Baguette", "Hallulla"], k=5)
        for pieza in piezas:
            piezas_de_pan.put(pieza)


def cliente(i: int) -> None:
    print(f"[Cobra {i}] ¡Quiero Queque!")
    mi_pan = piezas_de_pan.get()
    print(f"[Cobra {i}] ¡Saqué mi {mi_pan}!")
    piezas_de_pan.task_done()


thread_panadero = threading.Thread(target=panadero)
threads_clientes = []
```

```
for i in range(15):
    threads_clientes.append(threading.Thread(target=cliente,
args=(i, )))

thread_panadero.start()
for thread_cliente in threads_clientes:
    thread_cliente.start()

# Esperamos a que todos los threads finalicen para que esta celda
termine con su ejecución
for thread_cliente in threads_clientes:
    thread_cliente.join()
```

```
[Cliente 0] ¡Quiero pan!
[Cliente 1] ¡Quiero pan!
[Cliente 2] ¡Quiero pan!
[Cliente 3] ¡Quiero pan!
[Cliente 4] ¡Quiero pan!
[Cliente 5] ¡Quiero pan!
[Cliente 6] ¡Quiero pan!
[Cliente 7] ¡Quiero pan!
[Cliente 8] ¡Quiero pan!
[Cliente 9] ¡Quiero pan!
[Cliente 10] ¡Quiero pan!
[Cliente 11] ¡Quiero pan!
[Cliente 12] ¡Quiero pan!
[Cliente 13] ¡Quiero pan!
[Cliente 14] ¡Quiero pan!
[Panadero] Produje 5 piezas de pan en la partida 0
[Cliente 4] ¡Saqué mi Baguette!
[Cliente 1] ¡Saqué mi Marraqueta!
[Cliente 2] ¡Saqué mi Marraqueta!
[Cliente 3] ¡Saqué mi Hallulla!
[Cliente 0] ¡Saqué mi Baguette!
[Panadero] Produje 5 piezas de pan en la partida 1
[Cliente 7] ¡Saqué mi Marraqueta!
[Cliente 6] ¡Saqué mi Hallulla!
[Cliente 8] ¡Saqué mi Hallulla!
[Cliente 9] ¡Saqué mi Marraqueta!
[Cliente 5] ¡Saqué mi Marraqueta!
[Panadero] Produje 5 piezas de pan en la partida 2
[Cliente 12] ¡Saqué mi Baguette!
[Cliente 14] ¡Saqué mi Marraqueta!
[Cliente 13] ¡Saqué mi Hallulla!
[Cliente 11] ¡Saqué mi Marraqueta!
[Cliente 10] ¡Saqué mi Marraqueta!
```

En este ejemplo, solo 1 *thread* obtenía el recurso de la `Queue`, pero puede darse el caso que queremos que cada *thread* tenga su propio recurso. En particular, pensemos en un aviso del

curso. En este caso se manda un mensaje, pero queremos que todos los demás *threads* sean capaces de recibir ese recurso. Por lo tanto, en vez de usar una `Queue` compartida, podríamos usar una `Queue` por usuario, y cada uno de estos está atento a si su propia `Queue` tiene algún dato por procesar.

```
import threading
import queue
import random
import time


class Estudiante(threading.Thread):
    def __init__(self, nombre: str) -> None:
        # Enviamos el nombre con super() a la clase padre
        threading.Thread.__init__(self, name=nombre)

        # Creamos nuestra propia cola de correo
        self.cola_correos = queue.Queue()

    def notificar(self, aviso: str) -> None:
        self.cola_correos.put(aviso)

    def run(self) -> None:
        while True:
            aviso = self.cola_correos.get()
            print(f"[{self.name}] Ohhh tengo un correo")
            time.sleep(random.randint(0, 2))
            print(f"[{self.name}] Su contenido es: {aviso}")
            self.cola_correos.task_done()

class Canvas:
    def __init__(self) -> None:
        self.estudiantes = []

    def agregar_estudiante(self, estudiante: Estudiante) -> None:
        self.estudiantes.append(estudiante)

    def mandar_aviso(self, aviso: str) -> None:
        print("[CANVAS] Enviando un aviso a los estudiantes")
        for estudiante in self.estudiantes:
            estudiante.notificar(aviso)

canvas = Canvas()

nombres = ["Komi", "Naruto", "Luigi", "Gogeta", "Luffy"]
for nombre in nombres:
    estudiante = Estudiante(nombre)
    canvas.agregar_estudiante(estudiante)
    estudiante.start()
```

```
# Simulación del hilo principal que envía notificaciones
canvas.mandar_aviso("Aviso Semanal □")
canvas.mandar_aviso("Recuerden hacer la evaluación de Threading! ♀")
```

```
[CANVAS] Enviando un aviso a los estudiantes
[CANVAS] Enviando un aviso a los estudiantes
[Komi] Ohhh tengo un correo[Luigi] Ohhh tengo un correo
[Luigi] Su contenido es: Aviso Semanal □
[Luigi] Ohhh tengo un correo

[Gogeta] Ohhh tengo un correo
[Luffy] Ohhh tengo un correo
[Naruto] Ohhh tengo un correo
[Komi] Su contenido es: Aviso Semanal □[Luigi] Su contenido es:
Recuerden hacer la evaluación de Threading! ♀
[Luffy] Su contenido es: Aviso Semanal □
[Luffy] Ohhh tengo un correo
[Luffy] Su contenido es: Recuerden hacer la evaluación de Threading! ♀

[Komi] Ohhh tengo un correo
[Gogeta] Su contenido es: Aviso Semanal □[Naruto] Su contenido es:
Aviso Semanal □
[Naruto] Ohhh tengo un correo

[Gogeta] Ohhh tengo un correo
[Komi] Su contenido es: Recuerden hacer la evaluación de Threading! ♀
[Naruto] Su contenido es: Recuerden hacer la evaluación de
Threading! ♀
[Gogeta] Su contenido es: Recuerden hacer la evaluación de
Threading! ♀
```

Con `Queue` también podríamos comunicar *threads* para que hagan ciertas tareas. En ese caso, en vez de pasar objetos cualquiera podríamos pasar mensajes con cierto formato que todos puedan entender.

Simulaciones

Otra aplicación regular para el uso de *threads* es la simulación de entidades con comportamiento simultáneo. Esto, en conjunto a *locks* y señales, permite que se puedan modelar y simular situaciones únicas.

Por ejemplo, en el siguiente ejemplo se simula a un estudiante que estudia muy encarecidamente y programa línea por línea. Al mismo tiempo, se modela a su profesor que espera a que el estudiante tenga alguna duda durante su estudio, y solo en caso de que tenga una duda, este le responde:

```
from threading import Event, Thread
from time import sleep
from random import randint
```

```
class Estudiante(Thread): # Estudiante es un Thread

    def __init__(self, lineas_estudio: int, senal_duda: Event,
senal_resuelto: Event):
        super().__init__()
        self.max_lineas = lineas_estudio # Líneas de código que
programará
        self.senal_duda = senal_duda # Señal para notificar que
estudiante tiene una duda
        self.senal_resuelto = senal_resuelto # Señal para notificar
que profesor resolvió duda


    def run(self) -> None: # Simulación de comportamiento de
Estudiante

        print("[Estudiante] ¡A programar!")
        lineas = 0 # Comienza con archivo vacio.
        while lineas < self.max_lineas:
            sleep(1) # Se demora dos segundos en escribir una línea
            lineas += 1
            print("[Estudiante] --------- Una linea escrita\n")
            # Existe una probabilidad luego de cada línea de que tenga
una duda
            # A medida que escribe líneas, es menos probable
            if randint(1, self.max_lineas) >= lineas:
                print("[Estudiante] Tengo una duda profe :(")
                self.senal_duda.set() # Notifica que tuvo una duda.
                self.senal_resuelto.wait() # Espera a que duda se
resuelva
                self.senal_resuelto.clear() # Reinicia señal para
volver a usarse
                print("[Estudiante] Ahora entiendo profe, gracias :)\
n")
        print("[Estudiante] ¡Terminé de programar!")



class Profe(Thread): # Profe es un Thread

    def __init__(self, senal_duda: Event, senal_resuelto: Event) ->
None:
        super().__init__()
        self.senal_duda = senal_duda # Señal para notificar que
estudiante tiene una duda
        self.senal_resuelto = senal_resuelto # Señal para notificar
que profesor resolvió duda
        self.daemon = True # El profe debe ser daemon
```

```
    def run(self) -> None: # Simulación de comportamiento de Profe

        while True: # Espera indefinidamente por siempre. Como es
daemon, se detendrá con el resto del programa

            self.senal_duda.wait() # Espera a que alumne tenga una
duda.
            self.senal_duda.clear() # Reinicia señal para volver a
usarse.
            print("[Profesor] Mira, lo que pasa es que...")
            tiempo = randint(1, 4)
            sleep(tiempo) # Explica y se demora un poco
            print(f"[Profesor]... {tiempo} segundos después")
            self.senal_resuelto.set() # Notifica que la duda fue
resuelta.


senal_tengo_una_duda = Event()
senal_duda_resuelta = Event()

estudiante = Estudiante(10, senal_tengo_una_duda, senal_duda_resuelta)
profe = Profe(senal_tengo_una_duda, senal_duda_resuelta)

profe.start()
estudiante.start()
```

```
[Estudiante] ¡A programar!
[Estudiante] --------- Una linea escrita

[Estudiante] Tengo una duda profe :(
[Profesor] Mira, lo que pasa es que...
[Profesor]... 4 segundos después
[Estudiante] Ahora entiendo profe, gracias :)

[Estudiante] --------- Una linea escrita

[Estudiante] Tengo una duda profe :(
[Profesor] Mira, lo que pasa es que...
[Profesor]... 4 segundos después
[Estudiante] Ahora entiendo profe, gracias :)

[Estudiante] --------- Una linea escrita

[Estudiante] Tengo una duda profe :(
[Profesor] Mira, lo que pasa es que...
[Profesor]... 1 segundos después
[Estudiante] Ahora entiendo profe, gracias :)

[Estudiante] --------- Una linea escrita
```

```
[Estudiante] Tengo una duda profe :(
[Profesor] Mira, lo que pasa es que...
[Profesor]... 3 segundos después
[Estudiante] Ahora entiendo profe, gracias :)

[Estudiante] --------- Una linea escrita

[Estudiante] --------- Una linea escrita

[Estudiante] --------- Una linea escrita

[Estudiante] Tengo una duda profe :(
[Profesor] Mira, lo que pasa es que...
[Profesor]... 2 segundos después
[Estudiante] Ahora entiendo profe, gracias :)

[Estudiante] --------- Una linea escrita

[Estudiante] --------- Una linea escrita

[Estudiante] --------- Una linea escrita

[Estudiante] ¡Terminé de programar!
```